

数据库实验一之SQL练习 实验报告

时分 PB23151799

MySQL环境的安装配置

安装配置流程如下：

在官网下载MySQL最新版本（8.0.41-winx64）；

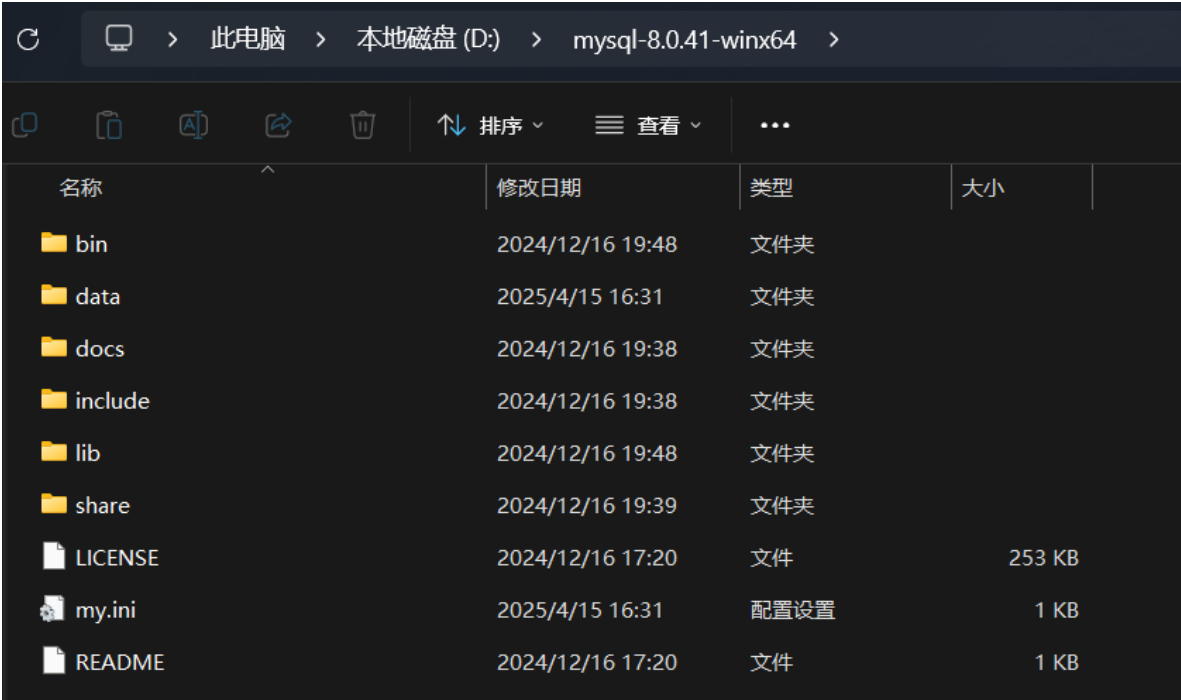
配置环境变量；

写入配置文件（my.ini）；

在CMD中完成MySQL环境初始化；

安装MySQL Workbench。

安装和配置结果如下：



名称	修改日期	类型	大小
bin	2024/12/16 19:48	文件夹	
data	2025/4/15 16:31	文件夹	
docs	2024/12/16 19:38	文件夹	
include	2024/12/16 19:38	文件夹	
lib	2024/12/16 19:48	文件夹	
share	2024/12/16 19:39	文件夹	
LICENSE	2024/12/16 17:20	文件	253 KB
my.ini	2025/4/15 16:31	配置设置	1 KB
README	2024/12/16 17:20	文件	1 KB

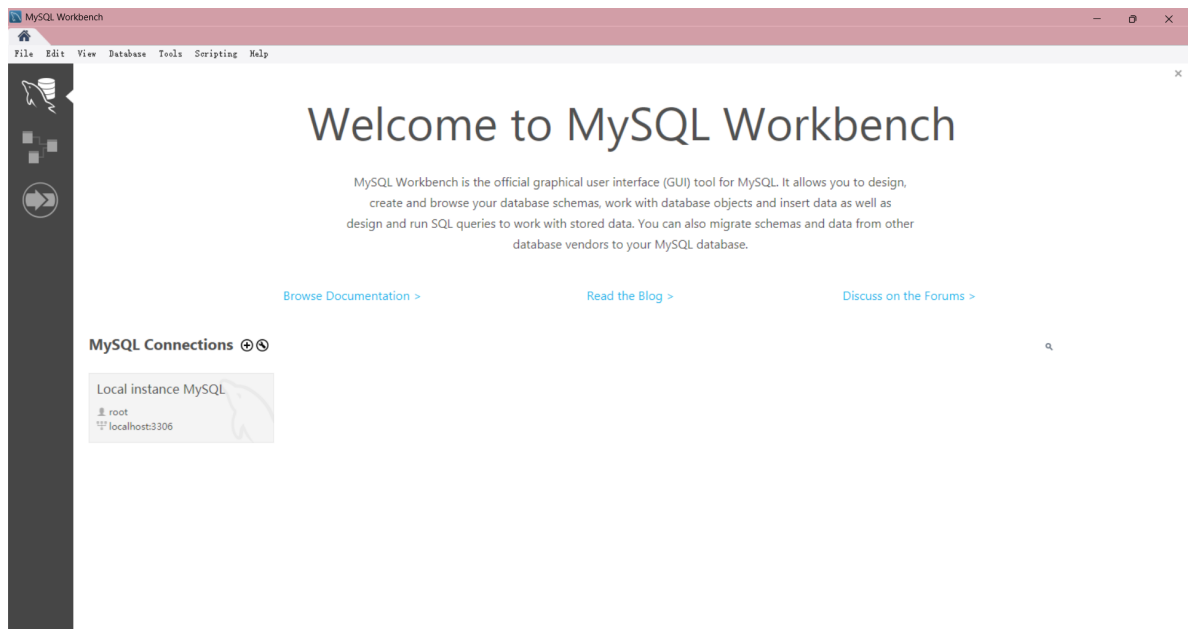
```
C:\Users\时分>mysql --version
mysql Ver 8.0.41 for Win64 on x86_64 (MySQL Community Server - GPL)

C:\Users\时分>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 33
Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> |
```



SQL练习

任务一

1. 创建模式：

```
create schema S_C_T_SC;
```

2. 创建表Student：

```
create table Student
(SNO char(11) primary key,
`NAME` varchar(4),      /*NAME是保留字*/
GENDER varchar(6),
AGE int,
DEPART varchar(3)
);
```

s_c_t_sc.student						
Info Columns Indexes Triggers Foreign keys Partitions Grants DDL						
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
NAME	varchar(4)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
GENDER	varchar(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
AGE	int		YES			select,insert,updat
DEPART	varchar(3)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat

3. 创建表Teacher：

```
create table Teacher
(TNO char(7) primary key,
`NAME` varchar(4),
GENDER varchar(6),
BIRTHDAY datetime,
POSITION varchar(25),
DEPART varchar(3)
);
```

s_c_t_sc.teacher						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants
DDL						
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
TNO	char(7)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
NAME	varchar(4)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
GENDER	varchar(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
BIRTHDAY	datetime		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
POSITION	varchar(25)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
DEPART	varchar(3)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat

4. 创建表Course:

```
create table Course
(CNO char(8) primary key,
`NAME` varchar(40),
CPNO char(8),
CCREDIT float,
TNO varchar(7),
foreign key (CPNO) references Course(CNO),
foreign key (TNO) references Teacher(TNO)
);
```

s_c_t_sc.course						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants
DDL						
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
CNO	char(8)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
NAME	varchar(40)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
CPNO	char(8)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
CCREDIT	float		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
TNO	varchar(7)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat

5. 创建表Score:

```
create table Score
(SNO char(11),
CNO char(8),
DEGREE int,
SEMESTER char(6),
primary key(SNO,CNO),
foreign key (SNO) references Student(SNO),
foreign key (CNO) references Course(CNO)
);
```

s_c_t_sc.score						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants
DDL						
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
CNO	char(8)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
DEGREE	int		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
SEMESTER	char(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat

任务二

修改基本表

1. 在学生表中增加一个新的属性列BIRTHYEAR（出生年份可为INT类型）

```
alter table Student add BIRTHYEAR int;
```

s_c_t_sc.score		s_c_t_sc.student					
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants	DDL
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges	
◆ SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat	
◆ NAME	varchar(4)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat	
◆ GENDER	varchar(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat	
◆ AGE	int		YES			select,insert,updat	
◆ DEPART	varchar(3)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat	
◆ BIRTHYEAR	int		YES			select,insert,updat	

2. 计算每个学生的出生年份（使用当前年份减去年龄）

```
update Student
set BIRTHYEAR = year(now()) - AGE;
```

Result Grid			 Filter Rows:			Edit:		
	SNO	NAME	GENDER	AGE	DEPART	BIRTHYEAR		
▶	PB230000001	YH	male	22	229	2003		
	PB230000002	ZY	male	22	11	2003		
	PB230000003	FWJ	male	21	12	2004		
	PB230000004	JTY	male	22	11	2003		
	PB230000005	YY	female	22	12	2003		
	PB230000006	HCC	male	21	229	2004		
	PB230000007	RZJ	male	20	11	2005		
	PB230000008	WCS	male	23	13	2002		
	PB230000009	ZMS	female	21	12	2004		
	PB230000010	WD	male	19	13	2006		
	PB230000011	BL	female	22	14	2003		
	PB230000012	ADN	male	23	10	2002		

3. 将每个学生的年龄减去2，并将年龄的类型从int改成char

```
update Student
set AGE = AGE-2;
alter table Student
modify AGE char(2);
```

Result Grid			 Filter Rows:	 Edit:			
	SNO	NAME	GENDER	AGE	DEPART	BIRTHYEAR	
▶	PB230000001	YH	male	20	229	2003	
	PB230000002	ZY	male	20	11	2003	
	PB230000003	FWJ	male	19	12	2004	
	PB230000004	JTY	male	20	11	2003	
	PB230000005	YY	female	20	12	2003	
	PB230000006	HCC	male	19	229	2004	
	PB230000007	RZJ	male	18	11	2005	
	PB230000008	WCS	male	21	13	2002	
	PB230000009	ZMS	female	19	12	2004	
	PB230000010	WD	male	17	13	2006	
	PB230000011	BL	female	20	14	2003	
	PB230000012	ADN	male	21	10	2002	

student						
s_c_t_sc.student						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants DDL
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
◇ SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,upda
◇ NAME	varchar(4)		YES	utf8mb3	utf8mb3_genera...	select,insert,upda
◇ GENDER	varchar(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,upda
◇ AGE	char(2)		YES	utf8mb3	utf8mb3_genera...	select,insert,upda
◇ DEPART	varchar(3)		YES	utf8mb3	utf8mb3_genera...	select,insert,upda
◇ BIRTHYEAR	int		YES			select,insert,upda

4. 将BIRTHYEAR列删除

```
alter table student
drop column BIRTHYEAR;
```

s_c_t_sc.student						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants DDL
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
◇ SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
◇ NAME	varchar(4)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
◇ GENDER	varchar(6)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
◇ AGE	char(2)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat
◇ DEPART	varchar(3)		YES	utf8mb3	utf8mb3_genera...	select,insert,updat

5. 创建一个学生选课课程数量表：student_course(SNO,NUM_COURSE)，两个属性分别表示授课学生学号，选课课程数量，其中SNO是主键。

```
create table student_course
(SNO char(11) primary key,
NUM_COURSE int,
foreign key(SNO) references Student(SNO)
);
```

s_c_t_sc.student						
s_c_t_sc.student_course						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants DDL
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
◇ SNO	char(11)		NO	utf8mb3	utf8mb3_genera...	select,insert,updat
◇ NUM_COURSE	int		YES			select,insert,updat

6. 用一条语句，结合表score记录，为表student中所有学生，在表student_course添加对应选课数量记录（用到查询，而不是手动数NUM插入）

```
insert into student_course(SNO, NUM_COURSE)
select Student.SNO, count(Score.CNO)
from Student left outer join Score on (Student.SNO = Score.SNO) /*使用左连接保
证包括Student中所有学生*/
group by Student.SNO;
```

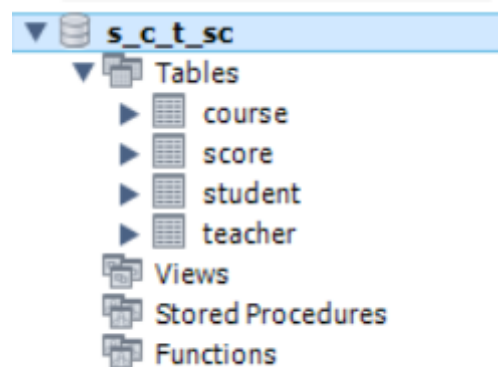
Result Grid			Filter Rows:
	SNO	NUM_COURSE	
▶	PB230000001	3	
	PB230000002	7	
	PB230000003	5	
	PB230000004	5	
	PB230000005	4	
	PB230000006	2	
	PB230000007	7	
	PB230000008	2	
	PB230000009	3	
	PB230000010	2	
	PB230000011	3	
	PB230000012	5	

7. 删除student_course中选修1门或者3门课的学生，然后删除student_course表

```
delete from student_course
where NUM_COURSE = 1 or NUM_COURSE = 3;
```

Result Grid			Filter Rows:
	SNO	NUM_COURSE	
▶	PB230000002	7	
	PB230000003	5	
	PB230000004	5	
	PB230000005	4	
	PB230000006	2	
	PB230000007	7	
	PB230000008	2	
	PB230000010	2	
	PB230000012	5	
	PB230000013	6	
	PB230000014	2	
	PB230000015	7	

```
drop table student_course;
```



8. 在Student表中添加学生姓名必须取唯一值的约束条件，然后再将这个约束条件删除

```
alter table Student
add constraint unique_name unique(`NAME`);
```

s_c_t_sc.student x

Info Columns Indexes Triggers Foreign keys Partitions Grants DDL

DDL for s_c_t_sc.student

```
1 CREATE TABLE `student` (  
2   `SNO` char(11) NOT NULL,  
3   `NAME` varchar(4) DEFAULT NULL,  
4   `GENDER` varchar(6) DEFAULT NULL,  
5   `AGE` char(2) DEFAULT NULL,  
6   `DEPARTMENT` varchar(3) DEFAULT NULL,  
7   PRIMARY KEY (`SNO`),  
8   UNIQUE KEY `unique_name` (`NAME`)  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3
```

```
alter table Student
drop constraint unique_name;
```

s_c_t_sc.student x

Info Columns Indexes Triggers Foreign keys Partitions Grants DDL

DDL for s_c_t_sc.student

```
1 CREATE TABLE `student` (  
2   `SNO` char(11) NOT NULL,  
3   `NAME` varchar(4) DEFAULT NULL,  
4   `GENDER` varchar(6) DEFAULT NULL,  
5   `AGE` char(2) DEFAULT NULL,  
6   `DEPARTMENT` varchar(3) DEFAULT NULL,  
7   PRIMARY KEY (`SNO`)  
8 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3
```

9. 删除Score表的复合主键，然后建立新的主键 (SNO,CNO,SEMESTER)

```
alter table Score
drop primary key,
add primary key (SNO,CNO,SEMESTER);
```

Indexes in Table

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	SNO, CNO, SEMESTER
☒	CNO	BTREE	NO	CNO

10. 修改Student表中DEPART列的列名为DEPARTMENT

```
alter table Student
rename column DEPART to DEPARTMENT;
```

Column	Type	Default Value	Nullable	Character Set	Collation	Privileges	Extra	Comments
SNO	char(11)		NO	utf8mb3	utf8mb3_general_ci	select,insert,update,references		
NAME	varchar(4)		YES	utf8mb3	utf8mb3_general_ci	select,insert,update,references		
GENDER	varchar(6)		YES	utf8mb3	utf8mb3_general_ci	select,insert,update,references		
AGE	char(2)		YES	utf8mb3	utf8mb3_general_ci	select,insert,update,references		
DEPARTMENT	varchar(3)		YES	utf8mb3	utf8mb3_general_ci	select,insert,update,references		

11. 在score表中删除DEPARTMENT为229的学生的课程记录，然后在score表中为每个学生删除其最低分的课程记录

```
delete from Score
where SNO in
(select SNO
from Student
where DEPARTMENT = 229);
```

	SNO	CNO	DEGREE	SEMESTER
▶	PB230000002	20230400	66	202202
	PB230000002	20230404	97	202302
	PB230000002	20230406	90	202201
	PB230000002	20230408	72	202401
	PB230000002	20230410	99	202301
	PB230000002	20230412	96	202302
	PB230000002	20230416	65	202202
	PB230000003	20230400	99	202302
	PB230000003	20230402	92	202402
	PB230000003	20230404	66	202401
	PB230000003	20230412	91	202202
	PB230000003	20230418	86	202401

```
/*MySQL不允许删除正在查询的表*/
/*需要利用查找得到临时表并内连接*/
delete s from Score as s
join (
    select SNO, min(DEGREE) as min_degree
    from Score
    group by (SNO)
) as temp on s.SNO = temp.SNO and s.DEGREE = temp.min_degree;
```


Result Grid   Filter Rows:				
	SNO	CNO	DEGREE	SEMESTER
▶	PB230000002	20230400	66	202202
	PB230000002	20230404	97	202302
	PB230000002	20230406	90	202201
	PB230000002	20230408	72	202401
	PB230000002	20230410	99	202301
	PB230000002	20230412	96	202302
	PB230000003	20230400	99	202302
	PB230000003	20230402	92	202402
	PB230000003	20230412	91	202202
	PB230000003	20230418	86	202401
	PB230000004	20230410	82	202302
	PB230000004	20230412	84	202202

12. 在成绩表 (Score) 中增加一个新的属性列GRADE (等级, CHAR类型), 并根据成绩 (Score) 填充等级: 90以上为'A', 80-89为'B', 70-79为'C', 60-69为'D', 60以下为'F'。提示, 可以使用CASE WHEN语句

```
alter table Score
add GRADE char(1);
update Score
set GRADE = (case
when DEGREE >= 90 then 'A'
when DEGREE >= 80 then 'B'
when DEGREE >= 70 then 'C'
when DEGREE >= 60 then 'D'
else 'A'
end);
```

Result Grid   Filter Rows: Edit: 					
	SNO	CNO	DEGREE	SEMESTER	GRADE
▶	PB230000002	20230400	66	202202	D
	PB230000002	20230404	97	202302	A
	PB230000002	20230406	90	202201	A
	PB230000002	20230408	72	202401	C
	PB230000002	20230410	99	202301	A
	PB230000002	20230412	96	202302	A
	PB230000003	20230400	99	202302	A
	PB230000003	20230402	92	202402	A
	PB230000003	20230412	91	202202	A
	PB230000003	20230418	86	202401	B
	PB230000004	20230410	82	202302	B
	PB230000004	20230412	84	202202	B

13. 在Student表中插入你自己的个人数据 (包含你的学号, 姓名缩写, 性别, 年龄, 系号), 注意: 为了防止学号撞车我们将学号扩充了一位, 所以你在插入的时候也可以将自己的学号扩充一位 (比如加一个0)

```
insert into Student(SNO, `NAME`, GENDER, AGE, DEPARTMENT)
values ('PB231517990', 'SF', 'male', '20', 229);
```

Result Grid					
Filter Rows:					
Edit:					
	SNO	NAME	GENDER	AGE	DEPARTMENT
	PB230000048	ZYH	male	21	10
	PB230000049	LX	female	19	229
	PB230000050	WXM	male	18	11
	PB230000051	ZYX	female	20	12
	PB230000052	LZ	male	19	13
	PB230000053	LYZ	female	18	14
	PB230000054	ZM	male	20	10
	PB230000055	LW	female	19	229
	PB230000056	ZZ	male	21	11
	PB231517990	SF	male	20	229
★	NULL	NULL	NULL	NULL	NULL

查询

1. 查询和你属于同一个系的学生学号和姓名(包括你本人)

```
select SNO, `NAME`
from Student
where DEPARTMENT =
(select DEPARTMENT
from Student
where SNO = 'PB231517990');
```

Result Grid		
Filter Rows:		
	SNO	NAME
▶	PB230000001	YH
	PB230000006	HCC
	PB230000020	XY
	PB230000021	LYH
	PB230000024	YHS
	PB230000025	YWB
	PB230000031	HM
	PB230000037	LY
	PB230000043	YM
	PB230000049	LX
	PB230000055	LW
	PB231517990	SF

2. 查询和你属于同一个系的学生学号和姓名(不包括你本人)

```
select SNO, `NAME`
from Student
where SNO <> 'PB231517990' AND DEPARTMENT =
(select DEPARTMENT
from Student
where SNO = 'PB231517990');
```

Result Grid			Filter Rows:
	SNO	NAME	
▶	PB230000001	YH	
	PB230000006	HCC	
	PB230000020	XY	
	PB230000021	LYH	
	PB230000024	YHS	
	PB230000025	YWB	
	PB230000031	HM	
	PB230000037	LY	
	PB230000043	YM	
	PB230000049	LX	
	PB230000055	LW	
	NULL	NULL	

3. 查询和YH同学不属于同一个系的学生学号和姓名。

```
select SNO, `NAME`
from Student
where DEPARTMENT <>
(select DEPARTMENT
from Student
where `NAME` = 'YH');
```

Result Grid			Filter Rows:
	SNO	NAME	
▶	PB230000002	ZY	
	PB230000003	FWJ	
	PB230000004	JTY	
	PB230000005	YY	
	PB230000007	RZJ	
	PB230000008	WCS	
	PB230000009	ZMS	
	PB230000010	WD	
	PB230000011	BL	
	PB230000012	ADN	
	PB230000013	HD	
	PB230000014	GNJ	

4. 查询和你以及YH同学都不在一个系的学生学号和姓名

```

select SNO, `NAME`
from Student
where DEPARTMENT <>
(select DEPARTMENT
from Student
where `NAME` = 'YH') AND DEPARTMENT <>
(select DEPARTMENT
from Student
where SNO = 'PB231517990');

```

Result Grid			Filter Rows
	SNO	NAME	
▶	PB230000002	ZY	
	PB230000003	FWJ	
	PB230000004	JTY	
	PB230000005	YY	
	PB230000007	RZJ	
	PB230000008	WCS	
	PB230000009	ZMS	
	PB230000010	WD	
	PB230000011	BL	
	PB230000012	ADN	
	PB230000013	HD	
	PB230000014	CHJ	

5. 查询229系老师的工号和姓名。

```

select TNO, `NAME`
from Teacher
where DEPART = '229';

```

Result Grid			Filter Rows
	TNO	NAME	
▶	TA90023	ZDH	
	TA90028	ZXY	
	TA90030	SY	
•	NULL	NULL	

6. 查询 11 系和 229 系教师的总人数。

```

select count(*)
from Teacher
where DEPART = '229' or DEPART = '11';

```

Result Grid		Filter Rows
	count(*)	
▶	7	

7. 查询年龄最大的学生的学号、姓名和年龄。

```
select SNO, `NAME`, AGE
from Student
where AGE =
(select max(AGE)
from Student); /*MySQL在执行MAX等函数时会自动进行类型转换（字符到数值）*/
```

Result Grid  Filter Rows:			
	SNO	NAME	AGE
▶	PB230000008	WCS	21
	PB230000012	ADN	21
	PB230000018	MY	21
	PB230000030	ZQ	21
	PB230000040	ZXY	21
	PB230000048	ZYH	21
	PB230000056	ZZ	21
●	NULL	NULL	NULL

8. 查询你的系中年龄最小的学生的学号、姓名和年龄

```
select SNO, `NAME`, AGE
from Student
where AGE =
(select min(AGE)
from Student
where DEPARTMENT =
(select DEPARTMENT
from Student
where SNO = 'PB231517990'));
```

Result Grid  Filter Rows:			
	SNO	NAME	AGE
▶	PB230000010	WD	17
	PB230000024	YHS	17
	PB230000035	LH	17
●	NULL	NULL	NULL

9. 查询选修 DB_Design 课程(不能通过课程号直接进行查询)且成绩在 80 分及以下的学生的学号、姓名和分数

```
/*解法一：使用嵌套查询*/
select Student.SNO, Student.`NAME`, Score.DEGREE
from Student, Score
where Student.SNO = Score.SNO and Score.DEGREE <= 80 and
Score.CNO =
(select CNO
from Course
where `NAME` = 'DB_Design');
```

```
/*解法二：使用连接查询*/
select Student.SNO, Student.`NAME`, Score.DEGREE
from Student, Score, Course
where Student.SNO = Score.SNO and Score.CNO = Course.CNO and
```

```
Course.`NAME` = 'DB_Design' and Score.DEGREE <= 80;
```

Result Grid  Filter Rows:			
	SNO	NAME	DEGREE
▶	PB230000013	HD	67
	PB230000029	LYF	69
	PB230000030	ZQ	77
	PB230000032	LL	65
	PB230000054	ZM	75

10. 查询选修过“ZDH”老师课程(不能通过课程号直接进行查询)的学生学号和姓名（去掉重复行）。

```
/*解法一：使用嵌套查询*/
select SNO, `NAME`
from Student
where SNO in
    (select SNO
     from Score
     where CNO in
         (select CNO
          from Course
          where TNO =
              (select TNO
               from Teacher
               where `NAME` = 'ZDH')));

/*解法二：使用连接查询*/
select distinct Student.SNO, Student.`NAME`
from Student, Course, Teacher, Score
where Student.SNO = Score.SNO and
Course.CNO = Score.CNO and
Course.TNO = Teacher.TNO and
Teacher.`NAME` = 'ZDH';

/*也可以结合嵌套和连接查询*/
```

Result Grid  Filter Rows:		
	SNO	NAME
▶	PB230000003	FWJ
	PB230000007	RZJ
	PB230000008	WCS
	PB230000011	BL
	PB230000013	HD
	PB230000027	ZXH
	PB230000028	WJY
	PB230000029	LYF
	PB230000030	ZQ
	PB230000032	LL
	PB230000033	XYH
	PB230000040	ZXY

11. 查询选过某课程的学生学号和分数，并按分数降序展示。（某课程是指 course 表中的某一课程名 NAME，你自行选择；分数指的是选的这门课的成绩，不包括选这门课的同学的其他成绩）。

```

/*解法一、使用连接查询*/
select Score.SNO, Score.DEGREE
from Score, Course
where Score.CNO = Course.CNO and Course.`NAME` = 'Artificial_Intelligence'
order by Score.DEGREE desc;

/*解法二、使用嵌套查询*/
select SNO, DEGREE
from Score
where CNO =
    (select CNO
     from Course
     where `NAME` = 'Artificial_Intelligence')
order by Score.DEGREE desc;

```

Result Grid  Filter Rows: 		
	SNO	DEGREE
▶	PB230000002	99
	PB230000033	95
	PB230000046	94
	PB230000035	88
	PB230000004	82
	PB230000030	75
	PB230000029	72
	PB230000015	70
	PB230000053	70
	PB230000012	68
	PB230000026	67
	PB230000038	67

12. 查询每门课的平均成绩，其中每行包含课程号、课程名和平均成绩（包括平均成绩为NULL，即该课没有成绩）。

```

select Course.CNO, Course.`NAME`, avg(Score.DEGREE)
from Course left outer join Score on (Score.CNO = Course.CNO) /*要求包含所有课程，使用外连接*/
group by (Course.CNO);

```

Result Grid   Filter Rows: Export: 			
	CNO	NAME	avg(Score.DEGREE)
▶	20230400	Linear_Algebra	79.5455
	20230402	DB_Design	84.4286
	20230404	Machine_Learning	83.5882
	20230406	Operating_System	81.1111
	20230408	Natural_Language_Processing	77.7273
	20230410	Artificial_Intelligence	78.0000
	20230412	Data_Mining	88.6000
	20230414	Signal_Control	86.6667
	20230416	Computer_Network	86.0714
	20230418	Pattern_Recognition	81.0714
	20230420	Deep_Learning	NULL

13. 查询学分大于 3(不包括3) 的课程的平均成绩，其中每行包含课程号、课程名和平均成绩（包括平均成绩为NULL，即该课没有成绩）

```
select Course.CNO, Course.`NAME`, avg(Score.DEGREE)
from Course left outer join Score on (Score.CNO = Course.CNO)
where Course.CCREDIT > 3
group by (Course.CNO);
```

Result Grid   Filter Rows: Ex			
	CNO	NAME	avg(Score.DEGREE)
▶	20230400	Linear_Algebra	79.5455
	20230402	DB_Design	84.4286
	20230404	Machine_Learning	83.5882
	20230406	Operating_System	81.1111
	20230412	Data_Mining	88.6000
	20230414	Signal_Control	86.6667
	20230420	Deep_Learning	NULL

14. 查询至少选修了工号TNO="TA90023"的老师（ZDH 老师）开设的所有课程的学生学号

```
select SNO
from Student
where not exists
(select *
from Course
where TNO = 'TA90023' and not exists
(select *
from Score
where Score.SNO = Student.SNO and Score.CNO = Course.CNO));
```


Result Grid	
SNO	
PB230000003	
PB230000040	
PB230000054	
HULL	

15. 查询每门课程的最高分和最低分，并计算其分数差。其中每行包含课程号、课程名和最高分、最低分和分数差。（课程无成绩的不用包括）。

```
select Score.CNO, Course.NAME, max(Score.DEGREE), min(Score.DEGREE),
max(Score.DEGREE)-min(Score.DEGREE)
from Score, Course
where Score.CNO = Course.CNO
group by (Score.CNO);
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
CNO	NAME	max(Score.DEGREE)	min(Score.DEGREE)	max(Score.DEGREE)-min(Score.DEGREE)
20230400	Linear_Algebra	99	66	33
20230404	Machine_Learning	99	67	32
20230406	Operating_System	90	73	17
20230408	Natural_Language_Processing	95	62	33
20230410	Artificial_Intelligence	99	67	32
20230412	Data_Mining	99	66	33
20230402	DB_Design	98	65	33
20230418	Pattern_Recognition	97	61	36
20230414	Signal_Control	99	77	22
20230416	Computer_Network	99	64	35

16. 查询存在考试成绩低于 78 分的学生学号，以及每个学生低于 78 分的课程的数量。

```
select SNO, count(*)
from Score
where DEGREE < 78
/*先执行where筛选出所有DEGREE<78的元组*/
group by SNO;
```

Result Grid		Filter Rows:
SNO	count(*)	
PB230000002	2	
PB230000005	1	
PB230000007	1	
PB230000012	3	
PB230000013	2	
PB230000015	3	
PB230000017	2	
PB230000019	2	
PB230000026	2	
PB230000029	4	
PB230000030	3	
PB230000032	3	

17. 查询所教过的课程中有学生考试成绩低于 72 分的教师的工号和姓名（去掉重复行）。

```
select distinct Teacher.TNO, Teacher.`NAME`
from Teacher, Course
where Teacher.TNO = Course.TNO and Course.CNO in
(select CNO
from Score
where DEGREE < 72);
```

	TNO	NAME
▶	TA90021	HMZ
	TA90023	ZDH
	TA90025	HTZ
	TA90027	XR
	TA90029	ZR

18. 查询选修大于 6 门课程的学生的学号、姓名

```
select SNO, `NAME`
from Student
where SNO in
(select SNO
from Score
group by (SNO)
having count(*) > 6);
```

/*在任务一T11中删除了每个学生的最低分记录，故学生最多只选修了6门课*/

	SNO	NAME
•	NULL	NULL

19. 查询至少选修了学号为SNO="PB230000002"的同学（ZY同学）所选全部课程的学生学号

```
select SNO
from Student
where not exists
(select *
from Score sc1
where SNO = 'PB230000002' and not exists
(select *
from Score sc2
where sc1.CNO = sc2.CNO and Student.SNO = sc2.SNO));
```

	SNO
▶	PB230000002
•	NULL

20. 查询student表中各个学生姓名与相应的平均成绩

```
select Student.`NAME`, avg(Score.DEGREE)
from Student left outer join Score on (Student.SNO = Score.SNO) /*有一些学生没有选课，使用外连接*/
group by Student.SNO;
```

Result Grid			Filter Rows:
	NAME	avg(Score.DEGREE)	
▶	YH	NULL	
	ZY	86.6667	
	FWJ	92.0000	
	JTY	86.3333	
	YY	83.3333	
	HCC	NULL	
	RZJ	91.0000	
	WCS	97.0000	
	ZMS	90.0000	
	WD	80.0000	
	BL	83.0000	
	ADN	78.0000	
	HD	76.4000	
	GNJ	99.0000	

21. 查询每个系的学生人数和每个系的平均分，其中每行包含系号、系的人数和平均成绩。（计算人数的时候需要包括那些没有成绩的同学，但是计算成绩的时候不需要包括这些同学）

```
select Student.DEPARTMENT, count(Student.SNO), avg(Score.DEGREE)
from Student left join Score on (Student.SNO = Score.SNO) /*有一些学生没有选课，使用外连接*/
group by Student.DEPARTMENT;
```

Result Grid				Filter Rows:	Export:
	DEPARTMENT	count(Student.SNO)	avg(Score.DEGREE)		
▶	229	12	NULL		
	11	40	84.6579		
	12	28	84.4643		
	13	16	86.4286		
	14	28	80.6071		
	10	21	77.7368		

22. 查询所有未选修 DB_Design 课程或者 Data_Mining 课程的学生姓名（去掉重复行）

```
select `NAME`
from Student
where SNO not in
(select SNO
from Score
where CNO in
(select CNO
from Course
where `NAME` = 'DB_Design' or `NAME` = 'Data_Mining'));
```

Result Grid			
NAME			
YH			
YY			
HCC			
GNJ			
XB			
LC			
TX			
MY			
XY			
LYH			
MSW			
HXY			
YHS			
YWR			

23. 查询各个课程的课程名及选该课的学生的平均年龄。（包括没有人选的课）

```
select Course.`NAME`, avg_age
from Course left join /*要求显示所有课程，使用外连接*/
(select Score.CNO, avg(cast(Student.AGE as signed)) as avg_age
/*在任务一T3中年龄类型被改为char，可以进行类型转换*/
/*AVG函数也会进行自动的类型转换*/
from Score, Student
where Score.SNO = Student.SNO
group by Score.CNO) as temp on (Course.CNO = temp.CNO);
```

Result Grid		Filter Rows:
NAME	avg_age	
Linear_Algebra	19.6364	
DB_Design	19.5714	
Machine_Learning	19.4118	
Operating_System	19.2222	
Natural_Language_Processing	19.0000	
Artificial_Intelligence	19.5385	
Data_Mining	19.2667	
Signal_Control	19.6667	
Computer_Network	18.8571	
Pattern_Recognition	19.4286	
Deep_Learning	NULL	

24. 查询选修了课程名中包含“Computer”课程的学生的学号和姓名

```
select Student.SNO, Student.`NAME`
from Student, Score
where Score.SNO = Student.SNO and Score.CNO in
(select CNO
from Course
where `NAME` like '%Computer%');
```

Result Grid   Filter Rows		
	SNO	NAME
▶	PB230000005	YY
	PB230000007	RZJ
	PB230000013	HD
	PB230000014	GNJ
	PB230000015	XB
	PB230000017	TX
	PB230000026	LXM
	PB230000027	ZXH
	PB230000032	LL
	PB230000035	LH
	PB230000038	WY
	PB230000039	LS
	PB230000051	ZYX
	PB230000053	LYZ

25. 设课程平均成绩为 x ，查询各个课程成绩处于 $[x-5, x+5]$ 区间的同学的成绩表，即包含 SNO、CNO、DEGREE

```
select Score.SNO, Score.CNO, Score.DEGREE
from Score join
  (select CNO, avg(DEGREE) as avg_degree
   from Score
   group by CNO) as course_avg on (Score.CNO = course_avg.CNO)
/*与CNO+平均分表连接*/
where Score.DEGREE <= course_avg.avg_degree + 5 and Score.DEGREE >=
course_avg.avg_degree - 5;
```

Result Grid   Filter Rows:			
	SNO	CNO	DEGREE
▶	PB230000015	20230400	78
	PB230000011	20230402	85
	PB230000027	20230402	85
	PB230000017	20230404	87
	PB230000028	20230404	87
	PB230000030	20230404	88
	PB230000012	20230406	77
	PB230000030	20230406	82
	PB230000035	20230406	80
	PB230000038	20230406	79
	PB230000013	20230408	79
	PB230000018	20230408	79
	PB230000042	20230408	77
	PB230000054	20230408	73
	PB230000004	20230410	82

26. 查询每门课程的课程号、课程名以及在这门课程上GRADE为A的学生人数，结果按人数降序排列。

```
select Course.CNO, Course.`NAME`, A_count
from Course join
(select CNO, count(*) as A_count
from Score
where GRADE = 'A'
group by CNO) as Course_account on (Course.CNO = Course_account.CNO)
order by A_count desc;
```

Result Grid   Filter Rows: Export			
	CNO	NAME	A_count
▶	20230412	Data_Mining	9
	20230402	DB_Design	7
	20230404	Machine_Learning	6
	20230416	Computer_Network	6
	20230400	Linear_Algebra	3
	20230408	Natural_Language_Processing	3
	20230410	Artificial_Intelligence	3
	20230414	Signal_Control	3
	20230418	Pattern_Recognition	3
	20230406	Operating_System	1

27. 查询包含 "_" (下划线) 且位 "_" 于后面的字符串包含"r"的所有课程的课程名。

```
select `NAME`
from Course
where `NAME` like '%\_%r%';
```

Result Grid   Filter Rows:	
	NAME
▶	Linear_Algebra
	Machine_Learning
	Natural_Language_Processing
	Signal_Control
	Computer_Network
	Pattern_Recognition
	Deep_Learning

28. 查询每个老师教的所有课程中最高的平均成绩，并展示出对应的老师工号和课程号

```
select Teacher.TNO, Course.CNO, avg_degree
from Teacher join
(Course join
(select CNO, avg(DEGREE) as avg_degree
from Score
group by (CNO)) as course_avg on (Course.CNO = course_avg.CNO))
on (Teacher.TNO = Course.TNO)
/*得到TNO-CNO-平均分表*/
where (Teacher.TNO, avg_degree) in
(select Teacher.TNO, max(avg_degree)
from Teacher join
(Course join (select CNO, avg(DEGREE) as avg_degree
```

```

from Score
group by (CNO)) as course_avg on (Course.CNO = course_avg.CNO))
on (Teacher.TNO = Course.TNO)
group by (Teacher.TNO));
/*得到TNO-最高平均分表*/

```

Result Grid				Filter Rows:
	TNO	CNO	avg_degree	
▶	TA90021	20230400	79.5455	
	TA90022	20230406	81.1111	
	TA90023	20230402	84.4286	
	TA90024	20230414	86.6667	
	TA90025	20230412	88.6000	
	TA90027	20230408	77.7273	
	TA90029	20230416	86.0714	

索引

1. 用 create 语句在 Student 表的名称 NAME 上建立普通索引 NAME_INDEX

```

create index NAME_INDEX
on Student(`NAME`);

```

Indexes in Table

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	SNO
☒	NAME_INDEX	BTREE	NO	NAME

2. 用 create 语句在 Course 表的课程号 CNO 上建立唯一索引 CNO_INDEX。

```

create unique index CNO_INDEX
on Course(CNO);

```

s_c_t_sc.course					
Info	Columns	Indexes	Triggers	Foreign keys	Partitions
Indexes in Table					
Visible	Key	Type	Unique	Columns	
☒	PRIMARY	BTREE	YES	CNO	
☒	CNO_INDEX	BTREE	YES	CNO	
☒	CPNO	BTREE	NO	CPNO	
☒	TNO	BTREE	NO	TNO	

- 用create 语句在 Score表中按学号升序，DEGREE成绩降序建立索引（即先按学号升序，然后按成绩降序）

```
create index SNO_DEGREE_INDEX
on Score(SNO asc, DEGREE desc);
```

Indexes in Table

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	SNO, CNO, SEMESTER
☒	CNO	BTREE	NO	CNO
☒	SNO_DEGREE_INDEX	BTREE	NO	SNO, DEGREE

- 用一条语句查询表 score 的索引。

```
show index from Score;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Nu
score	0	PRIMARY	1	SNO	A	39			
score	0	PRIMARY	2	CNO	A	127			
score	0	PRIMARY	3	SEMESTER	A	127			
score	1	CNO	1	CNO	A	10			
score	1	SNO_DEGREE_INDEX	1	SNO	A	39			
score	1	SNO_DEGREE_INDEX	2	DEGREE	D	116			YES

- 删除Course 表的课程号 CNO 上建立的唯一索引 CNO_INDEX

```
drop index CNO_INDEX on Course;
```

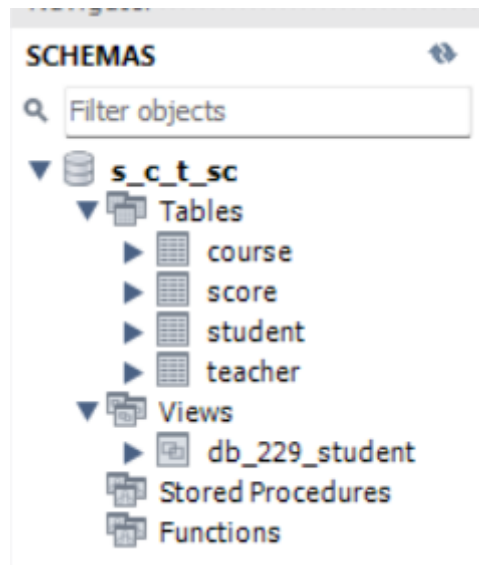
Indexes in Table

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	CNO
☒	CPNO	BTREE	NO	CPNO
☒	TNO	BTREE	NO	TNO

视图

- 建立 229 系的学生视图（db_229_student），属性与 student 表一样，并要求对该视图进行修改和插入操作时仍需保证该视图只有 229 系的学生


```
create view db_229_student
as
select *
from Student
where DEPARTMENT = 229
with check option;
```



2. 将 229 系学生视图 (db_229_student) 中学号为“PB210000031”的学生姓名和学号改为你的姓名和学号。

```
update db_229_student
set `NAME` = 'SF', SNO = 'PB231517990'
where SNO = 'PB210000031';
/*视图中不存在该学号*/
```

Result Grid					
Filter Rows:					
	SNO	NAME	GENDER	AGE	DEPARTMENT
	PB230000001	YH	male	20	229
	PB230000006	HCC	male	19	229
	PB230000020	XY	female	19	229
	PB230000021	LYH	male	20	229
	PB230000024	YHS	female	17	229
	PB230000025	YWB	male	20	229
▶	PB230000031	HM	female	20	229
	PB230000037	LY	female	18	229
	PB230000043	YM	female	19	229
	PB230000049	LX	female	19	229

3. 在 229 系学生视图 (db_229_student) 中找出年龄小于 21 岁的学生, 包含 SNO、NAME、GENDER

```
select SNO, `NAME`, GENDER
from db_229_student
where AGE < 21;
```

Result Grid				Filter Rows:
	SNO	NAME	GENDER	
▶	PB230000001	YH	male	
	PB230000006	HCC	male	
	PB230000020	XY	female	
	PB230000021	LYH	male	
	PB230000024	YHS	female	
	PB230000025	YWB	male	
	PB230000031	HM	female	
	PB230000037	LY	female	
	PB230000043	YM	female	
	PB230000049	LX	female	

4. 向 student 表中插入一名“学号 SA242290001, 姓名 QWE, 性别male, 229系, 年龄22”的学生。然后查询视图 db_229_student 的所有学生, 验证其是否更新。

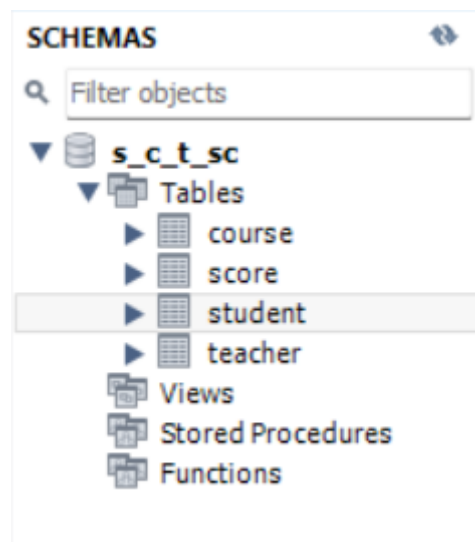
```
insert into db_229_student
values('SA242290001','QWE','male','22',229);

select *
from db_229_student;
```

Result Grid						Filter Rows:	Export:	Wra
	SNO	NAME	GENDER	AGE	DEPARTMENT			
	PB230000021	LYH	male	20	229			
	PB230000024	YHS	female	17	229			
	PB230000025	YWB	male	20	229			
	PB230000031	HM	female	20	229			
	PB230000037	LY	female	18	229			
	PB230000043	YM	female	19	229			
	PB230000049	LX	female	19	229			
	PB230000055	LW	female	19	229			
	PB231517990	SF	male	20	229			
	SA242290001	QWE	male	22	229			

5. 删除视图 db_229_student。

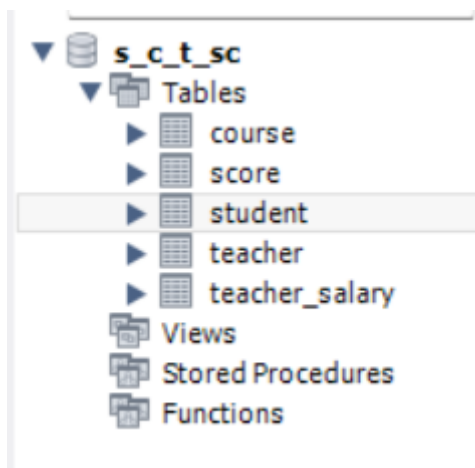
```
drop view db_229_student;
```



触发器

1. 创建关系表：teacher_salary(TNO, SAL)，其中 TNO 是教师工号（主键），SAL 是教师工资（类型float）。

```
create table teacher_salary
(TNO char(7) primary key,
SAL float
);
```



2. 定义一个 BEFORE 行级触发器，为关系表 teacher_salary 定义完整性规则：“表中出现的工号必须也出现在teacher 表中，否则报错”。

```
delimiter //
create trigger TS_T1
before update on teacher_salary
for each row
begin
    if new.TNO not in (select TNO from Teacher) then
        signal sqlstate '10000' set message_text = 'TNO is not in Teacher!'
    ;
    end if;
end //
create trigger TS_T2
before insert on teacher_salary
for each row
begin
```

```

if new.TNO not in (select TNO from Teacher) then
    signal sqlstate '10000' set message_text = 'TNO is not in Teacher!';
end if;
end //
delimiter ;

```

Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants	DDL
Name	Event	Timing	Created	SQL Mode	Definer	Client	
TS_T2	INSERT	BEFORE	2025-04-17 18:0...	ONLY_FULL_GR...	root@localhost	utf8n	
TS_T1	UPDATE	BEFORE	2025-04-17 18:0...	ONLY_FULL_GR...	root@localhost	utf8n	

测试：插入

```

386 • insert into teacher_salary
387 values ('TA88888', 1000);

```

#	Time	Action	Message	Duration / Fetch
1	18:06:46	insert into teacher_salary values ('TA88888', 1000)	Error Code: 1644. TNO is not in Teacher!	0.000 sec

测试：修改

```

386 • insert into teacher_salary
387 values ('TA90021', 30000);
388 • update teacher_salary
389 set TNO = 'TA88888'
390 where TNO = 'TA90021';

```

#	Time	Action	Message	Duration / Fetch
1	18:08:15	update teacher_salary set TNO = 'TA88888' where T...	Error Code: 1644. TNO is not in Teacher!	0.000 sec

- 定义一个 BEFORE 行级触发器，为关系表 teacher_salary 定义完整性规则：
“Instructor/Associate Professor/Professor 的工资不能低于4000/7000/10000，且不能高于7000/10000/13000，如果低于则改为4000/7000/10000”，如果高于则改成7000/10000/13000。

```

delimiter //

create trigger TS_T3
before update on teacher_salary
for each row
begin
    if new.TNO in (select TNO from Teacher where POSITION = 'Instructor')
    then
        if new.SAL < 4000 then
            set new.SAL = 4000;
        elseif new.SAL > 7000 then
            set new.SAL = 7000;
        end if;
    elseif new.TNO in (select TNO from Teacher where POSITION = 'Associate
Professor') then
        if new.SAL < 7000 then
            set new.SAL = 7000;
        elseif new.SAL > 10000 then
            set new.SAL = 10000;
        end if;
    end if;
end //

```

```

else
    if new.SAL < 10000 then
        set new.SAL = 10000;
    elseif new.SAL > 13000 then
        set new.SAL = 13000;
    end if;
end if;
end //

create trigger TS_T4
before insert on teacher_salary
for each row
begin
    if new.TNO in (select TNO from Teacher where POSITION = 'Instructor')
then
        if new.SAL < 4000 then
            set new.SAL = 4000;
        elseif new.SAL > 7000 then
            set new.SAL = 7000;
        end if;
    elseif new.TNO in (select TNO from Teacher where POSITION = 'Associate
Professor') then
        if new.SAL < 7000 then
            set new.SAL = 7000;
        elseif new.SAL > 10000 then
            set new.SAL = 10000;
        end if;
    else
        if new.SAL < 10000 then
            set new.SAL = 10000;
        elseif new.SAL > 13000 then
            set new.SAL = 13000;
        end if;
    end if;
end //

delimiter ;

```

teacher						
teacher_salary						
s_c_t.sc.teacher_salary						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants
Name	Event	Timing	Created	SQL Mode	Definer	Client
TS_T2	INSERT	BEFORE	2025-04-17 18:0...	ONLY_FULL_GR...	root@localhost	utf8n
TS_T4	INSERT	BEFORE	2025-04-17 18:1...	ONLY_FULL_GR...	root@localhost	utf8n
TS_T1	UPDATE	BEFORE	2025-04-17 18:0...	ONLY_FULL_GR...	root@localhost	utf8n
TS_T3	UPDATE	BEFORE	2025-04-17 18:1...	ONLY_FULL_GR...	root@localhost	utf8n

测试：插入('TA90024',30000), 'TA90024'是Associate Professor, 故SAL被自动修改为10000.

```

440 • insert into teacher_salary
441 values('TA90024', 30000);
442 /*TA90024 is Associate Professor*/

```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	18:19:54	insert into teacher_salary values('TA90024', 30000)	1 row(s) affected	0.094 sec

	TNO	SAL
▶	TA90024	10000
★	NULL	NULL

测试：修改('TA90024',10000)为('TA90030',1), 'TA90030'是Professor，故SAL被自动修改为10000。

```

444 • update teacher_salary
445     set TNO = 'TA90030', SAL = 1
446     where TNO = 'TA90024';
447     /*TA90024 is Professor*/

```

#	Time	Action	Message	Duration / Fetch
✓ 1	21:38:14	update teacher_salary set TNO = 'TA90030', SAL = 1 ...	1 row(s) affected Rows matched: 1 Changed: 1 War...	0.016 sec

	TNO	SAL
▶	TA90030	10000
★	NULL	NULL

4. 删除刚刚创建的所有触发器。

```

drop trigger TS_T1;
drop trigger TS_T2;
drop trigger TS_T3;
drop trigger TS_T4;

```

s_c_t_sc.teacher_salary						
Info	Columns	Indexes	Triggers	Foreign keys	Partitions	Grants
Name	Event	Timing	Created	SQL Mode	Definer	Client

空值

1. 将 score 表中的DB_Design课程成绩设为空值，然后在 score 表查询选修此课学生学号和分数，并按分数升序展示。观察 NULL 在 MySQL 中的大小是怎样的？

```

update Score
set DEGREE = NULL
where CNO =
    (select CNO
     from Course
     where `NAME` = 'DB_Design');

/*查询选修此刻学生学号和其选修过的所有课程分数，否则升序展示无法显示出NULL的大小*/
select SNO, DEGREE
from Score
where SNO in
    (select SNO
     from Score
     where CNO =
         (select CNO

```

```

from Course
where `NAME` = 'DB_Design'))
order by DEGREE asc;

```

Result Grid			Filter Rows:
	SNO	DEGREE	
▶	PB230000028	NULL	
	PB230000003	NULL	
	PB230000032	NULL	
	PB230000051	NULL	
	PB230000007	NULL	
	PB230000040	NULL	
	PB230000033	NULL	
	PB230000030	NULL	
	PB230000008	NULL	
	PB230000011	NULL	
	PB230000013	NULL	
	PB230000029	NULL	
	PB230000027	NULL	
	PB230000054	NULL	
	PB230000032	62	

即NULL在MySQL中被设为最小值。

开放题

请自己设计两个题目并给出sql代码实现。

1. 基于Score.csv文件建立Score_new表（避免之前题目中删除的分数数据影响），添加属性GPA，并基于科大的标准计算所有记录的GPA。

```

create table Score_new
(SNO char(11),
CNO char(8),
DEGREE int,
SEMESTER char(6),
primary key(SNO,CNO),
foreign key (SNO) references Student(SNO),
foreign key (CNO) references Course(CNO)
);

alter table Score_new
add GPA float;

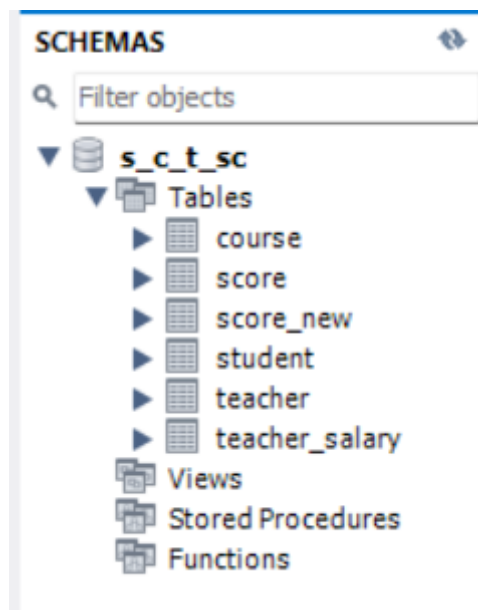
update Score_new
set GPA = (case
when DEGREE >= 95 then 4.3
when DEGREE >= 90 then 4.0
when DEGREE >= 85 then 3.7
when DEGREE >= 82 then 3.3
when DEGREE >= 78 then 3.0
when DEGREE >= 75 then 2.7
when DEGREE >= 72 then 2.3
when DEGREE >= 68 then 2

```

```

when DEGREE >= 65 then 1.7
when DEGREE >= 64 then 1.5
when DEGREE >= 61 then 1.3
when DEGREE >= 60 then 1
else 0
end);

```



	SNO	CNO	DEGREE	SEMESTER	GPA
▶	PB230000001	20230408	72	202401	2.3
	PB230000001	20230416	88	202202	3.7
	PB230000001	20230418	79	202202	3
	PB230000002	20230400	66	202202	1.7
	PB230000002	20230404	97	202302	4.3
	PB230000002	20230406	90	202201	4
	PB230000002	20230408	72	202401	2.3
	PB230000002	20230410	99	202301	4.3
	PB230000002	20230412	96	202302	4.3
	PB230000002	20230416	65	202202	1.7
	PB230000003	20230400	99	202302	4.3
	PB230000003	20230402	92	202402	4
	PB230000003	20230404	66	202401	1.7
	PB230000003	20230412	91	202202	4
	PB230000003	20230418	86	202401	3.7

2. 为Student表添加属性总学分CREDIT、加权平均分AVG_SCORE、总绩点GPA，并基于Student、Course、Score_new表的内容填充，加权平均分和GPA的计算按照科大的评分标准。最后展示每个专业的GPA第一名（输出Student表里的各项数据）。

```

alter table Student
add CREDIT float,
add AVG_SCORE float,
add GPA float;

/*计算总学分*/
update Student
set CREDIT =

```



```
(select sum(CREDIT)
from Course, Score_new
where Course.CNO = Score_new.CNO and Score_new.SNO = Student.SNO);

/*计算加权平均分*/
update Student
set AVG_SCORE =
(select sum(Score_new.DEGREE * Course.CREDIT)/Student.CREDIT
from Course, Score_new
where Course.CNO = Score_new.CNO and Score_new.SNO = Student.SNO);

/*计算GPA*/
update Student
set GPA =
(select sum(Score_new.GPA * Course.CREDIT)/Student.CREDIT
from Course, Score_new
where Course.CNO = Score_new.CNO and Score_new.SNO = Student.SNO);
```

Result Grid

 Filter Rows:

Edit:







Export/Import:





	SNO	NAME	GENDER	AGE	DEPARTMENT	CREDIT	AVG_SCORE	GPA
▶	PB230000001	YH	male	20	229	8	80.1875	3.04375
	PB230000002	ZY	male	20	11	24	84.1667	3.28333
	PB230000003	FWJ	male	19	12	17.5	86.6	3.5
	PB230000004	JTY	male	20	11	16	84.5	3.37188
	PB230000005	YY	female	20	12	14.5	77.9655	2.78966
	PB230000006	HCC	male	19	229	8	80	3
	PB230000007	RZJ	male	18	11	23.5	86.4894	3.47234
	PB230000008	WCS	male	21	13	7.5	90.6	3.98
	PB230000009	ZMS	female	19	12	10.5	78	2.86667
	PB230000010	WD	male	17	13	6.5	74.9231	2.53846
	PB230000011	BL	female	20	14	10	77.15	2.745
	PB230000012	ADN	male	21	10	17	76.0882	2.56765
	PB230000013	HD	male	20	14	19.5	74.5897	2.54872
	PB230000014	GNJ	male	19	11	5.5	90.8182	3.70909
	PB230000015	XB	female	20	12	23	75.5	2.64348

```
/*展示每个专业的GPA第一名*/
select *
from Student
where (DEPARTMENT, GPA) in
      (select DEPARTMENT, max(GPA)
       from Student
       group by DEPARTMENT);
```

[illegible]